

Tendências de Motores de Jogos

Semana 1

O que é uma Game Engine? Actor e Component

Disciplina: Tendências de Motores de Jogos (IN46A)

Instituição: IFMS — Campus Dourados

Curso: Tecnologia em Jogos Digitais

Módulo 1 — Aprender a Ferramenta

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

OBJETIVOS

- Compreender o que é uma game engine, antes de qualquer botão
- Reconhecer a organização do Unreal Editor como instância de conceitos universais
- Compreender Actor e Component como unidade de composição
- Iniciar a estrutura do projeto único do semestre: o Vertical Slice

Como a semana está organizada

- **Encontro 1**
O que é uma game engine + tour pelo Unreal Editor + estrutura de pastas do Vertical Slice
- **Encontro 2**
Actor e Component como composição + criação guiada + desafio de variação

Tendências de Motores de Jogos

ENCONTRO 1

O que é uma Game Engine?

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Aprender Unreal é a mesma coisa que aprender a fazer jogos?

Pense na diferença entre a ferramenta e o que se constrói com ela.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Engine, Editor e Jogo

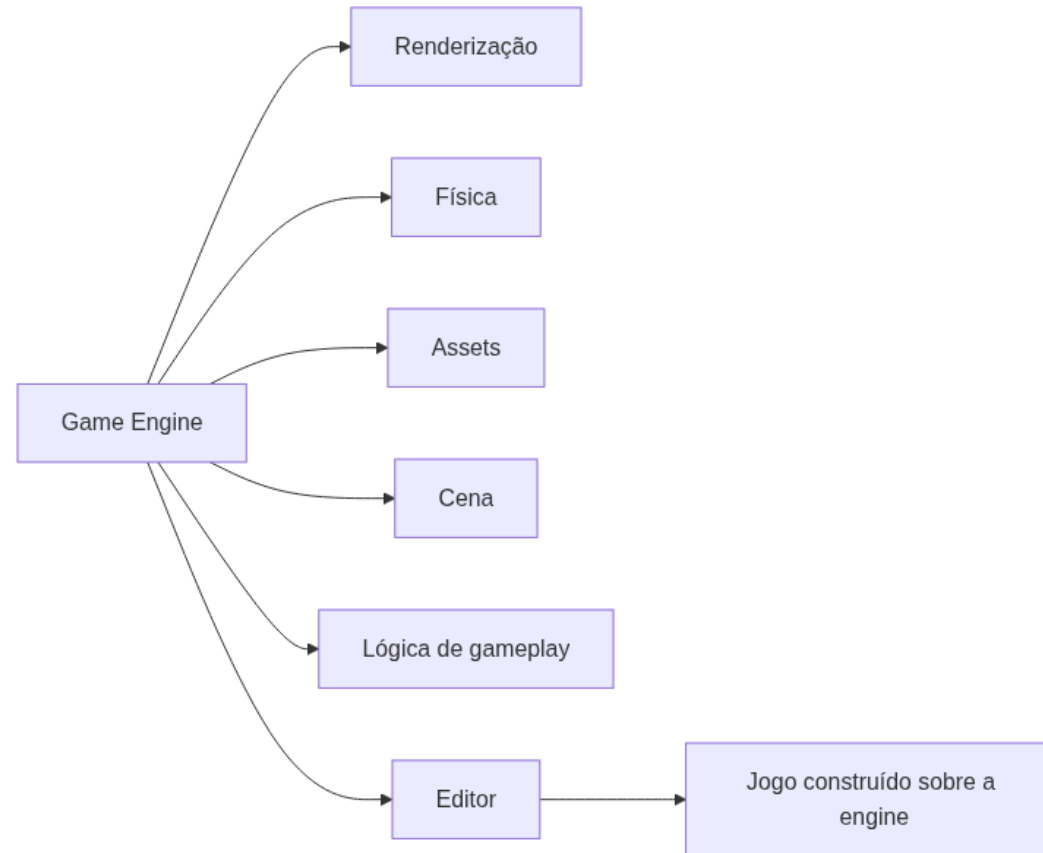
- **Engine** — software que resolve renderização, física, assets, cena e lógica de forma reutilizável
- **Editor** — interface que expõe essas ferramentas ao desenvolvedor
- **Jogo** — o conteúdo e a lógica específicos construídos sobre a engine



Unreal, Unity, Godot e O3DE resolvem o mesmo conjunto de problemas, cada uma à sua maneira.

Tendências de Motores de Jogos

Onde a Unreal se encaixa



Unreal × Unity — primeira aproximação

Unreal Editor

Viewport, Content Browser, Outliner, Details

Unity Editor

Scene View, Project Window, Hierarchy, Inspector

Demonstração: tour pelo Unreal Editor

O professor vai mostrar, ao vivo, as quatro áreas principais do editor e a função de cada uma — antes de qualquer ação prática dos estudantes.



O objetivo não é decorar botões, é reconhecer o papel de cada área.

Imagem sugerida

Objetivo: mostrar a disposição geral do Unreal Editor com as quatro áreas principais destacadas.

Enquadramento: captura de tela cheia do editor logo após a criação do projeto Third Person, Viewport ao centro.

Elementos presentes: Viewport, Content Browser, Outliner, Details, cada um com contorno colorido e rótulo.

Destaque visual: contornos coloridos ao redor de cada área.

Legenda sugerida: "As quatro áreas principais do Unreal Editor: Viewport, Content Browser, Outliner e Details."

Arquitetura: a estrutura de pastas do Vertical Slice

- Blueprints/ — Characters, Interactables, Framework, Components
- Maps/ — Exploration, Dungeon
- Environment/ , Characters/ , UI/ , Materials/ , Audio/ , Data/ , Textures/ , Meshes/

NA INDÚSTRIA

A mesma lógica de organizar por função, não por tipo de arquivo, é padrão em qualquer estúdio profissional.

Boas práticas de organização

✓ BOAS PRÁTICAS

- Agrupar por função, nunca por tipo de arquivo solto
- Usar prefixos desde o início (BP_ , Map_)
- Nunca renomear ou mover arquivos pelo explorador do sistema operacional

Laboratório do Encontro 1

Criar o projeto Third Person em Blueprint e organizar a estrutura de pastas completa do Vertical Slice, replicando o modelo demonstrado.

OBJETIVOS

Critério de sucesso: projeto criado, pastas organizadas, nenhum asset solto na raiz de `Content/`.

Tendências de Motores de Jogos

Fechando o Encontro 1

- Engine ≠ Editor ≠ Jogo
- Projeto do Vertical Slice criado e organizado
- Próximo encontro: Actor e Component

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos

ENCONTRO 2

Actor e Component

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Como dar forma e comportamento a um objeto sem uma árvore rígida de heranças?

Pense em quantas classes seriam necessárias se cada combinação de "tem luz", "tem colisão", "tem malha" exigisse uma subclasse nova.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Composição sobre Herança

- Comportamento montado anexando peças reutilizáveis, não por herança de classes
- Cada peça resolve **uma** responsabilidade isolada
- O mesmo Component pode ser reaproveitado em Actors completamente diferentes



É o mesmo problema que toda engine moderna precisa resolver.

Actor: o contêiner universal

- Unidade básica de qualquer objeto que existe em um nível
- Sem Components, é apenas um ponto no espaço
- Ganha forma, colisão e comportamento anexando Components

Component: responsabilidade isolada

- **Static Mesh Component** — forma visual
- **Collision** — interação física com o mundo
- **Light Component** — emissão de luz



DICA

Nenhum Component sabe da existência dos outros.

Actor/Component × GameObject/Component

Unreal

Actor + Components (Static Mesh, Collision, Light)

Unity

GameObject + Components (MeshRenderer, Collider, Light)

Demonstração: o que será construído

Um Actor Blueprint (`BP_TesteComposicao`) com Static Mesh, Point Light e colisão, testado em modo Play contra o personagem do template.

Resultado esperado: malha visível, luz emitida, colisão bloqueando o jogador.

Imagem sugerida

Objetivo: mostrar o painel de Components do Blueprint com três Components anexados.

Enquadramento: editor de Blueprint com o painel de Components à esquerda expandido.

Elementos presentes: DefaultSceneRoot como pai de Static Mesh, Point Light e colisão.

Destaque visual: estrutura em árvore do painel, cada item rotulado por função.

Legenda sugerida: "Um Actor composto por três Components independentes: malha, colisão e luz."

Boas práticas

✓ BOAS PRÁTICAS

- Nomear cada Component de forma descritiva (`MalhaPrincipal` , não `StaticMesh1`)
- Compilar antes de testar em modo Play
- Comentar decisões de composição direto no Blueprint

Laboratório do Encontro 2

Replicar o `BP_Testecomposicao` demonstrado: Actor puro com Static Mesh, Point Light e colisão funcional.

OBJETIVOS

Critério de sucesso: Actor bloqueia o personagem em modo Play e emite luz visível.

Desafio: variação própria

Adicionar um Component adicional que produza um comportamento visual diferente do demonstrado — sem gabarito único.

ATENÇÃO

Resolva por composição (novo Component), nunca criando uma nova classe de Actor.

Resumo da Semana 1

- Engine, Editor e Jogo são camadas diferentes
- Projeto do Vertical Slice criado e organizado
- Actor e Component como composição sobre herança
- Cada grupo produziu uma variação própria de Component

Próximos passos



Na Semana 2, o `BP_Player` nasce de um **Character** — subclasse de Actor com Components de movimentação já embutidos.

Leitura recomendada: Actors in Unreal Engine · Components in Unreal Engine (Epic Games Documentation).